

Taller: Introducción a React Native

Entornos de desarrollo, navegación entre pantallas y configuración base de Firebase

Duración: 90 minutos | **Modalidad:** individual | **Entrega:** enlace del repositorio en Virtual TdeA

Objetivo. Comprender qué exige el entorno de desarrollo móvil según el sistema operativo, distinguir React Native de React para web, manejar la navegación entre pantallas y dejar configurada la conexión con Firebase en una aplicación funcional.

Requisitos previos. Node.js LTS instalado, un editor de código, un dispositivo físico con Expo Go o un emulador configurado, y una cuenta de Google para la consola de Firebase. La documentación oficial está permitida y es la fuente recomendada durante todo el taller.

Carácter individual. Este taller se desarrolla y se entrega de forma individual. Puede consultar con sus compañeros y con el docente para resolver dudas puntuales, pero el código y las respuestas deben ser de su propia autoría. Se revisará la similitud entre las entregas y el historial de commits del repositorio.

Momento	Tiempo	Peso	Contenido
Parte 1 — Preguntas 1 a 4	30 min	40%	Entornos, fundamentos, navegación y Firebase
Parte 2 — Pregunta 5	55 min	60%	Aplicación de tres pantallas conectada a Firestore
Cierre	5 min	—	Verificación, subida a GitHub y entrega

Parte 1 — Componente teórico

40% · 30 minutos

Responda con sus propias palabras. Máximo 8 líneas por literal. Las respuestas van en un archivo `RESPUESTAS.md` dentro del repositorio.

PREGUNTA 1Entorno de desarrollo y sistema operativo

a) Complete la tabla explicando qué papel cumple cada pieza del entorno de desarrollo móvil:

Herramienta	¿Qué papel cumple?
Node.js y npm	Node.js es el entorno de ejecución que permite ejecutar a JavaScript fuera del navegador, además de proporcionar el entorno necesario para ejecutar herramientas necesarias durante el desarrollo. Por otro lado, npm (node package manager) es el administrador de paquetes para Node.js, encargado de instalar, actualizar y administrar los paquetes y dependencias de un proyecto.
Metro bundler	Metro bundler es el empaquetador de JavaScript utilizado por React Native, se encarga de analizar, resolver y empaquetar el código y las dependencias para que puedan ejecutarse en la aplicación o emulador.
JDK y Android SDK	JDK (Java Development Kit) es el conjunto de herramientas necesario para desarrollar y copilar programas que utilicen Java y que en Android es utilizado por herramientas como Gradle para construir la aplicación. Por otro lado, el Android SDK (Android software Developmet Kit) es el conjunto de herramientas, Apis y componentes que proporciona Android para desarrollar, copilar, depurar y ejecutar aplicaciones para dispositivos Android.
Xcode	Xcode es el entorno de desarrollo (IDE) oficial de Apple para desarrollar, copilar, probar y depurar aplicaciones para sus dispositivos y aplicaciones especialmente iOS y macOS. En el contexto de React Native/ Expo, Xcode es muy importante para el desarrollo y ejecución de aplicaciones para iOS.
Expo Go	Expo Go es una aplicación móvil que permite ejecutar, visualizar y probar proyectos de React Native desarrollados con Expo directamente en un dispositivo físico, facilitando el proceso de desarrollo sin necesidad de realizar una compilación nativa completa. Teniendo en cuenta que Expo Go no reemplaza a Android SDK, JDK, o Xcode.

b) Un estudiante con **Windows** y otro con **Linux** quieren compilar la misma aplicación para **iOS**. Explique por qué ninguno de los dos puede hacerlo en su propia máquina y mencione dos alternativas reales para que igual logren probar y publicar la app.

- Los estudiantes de Windows y Linux no pueden compilar directamente la aplicación para iOS en sus máquinas porque **Xcode y el SDK de iOS requieren macOS**. Para lograrlo pueden **utilizar un equipo Mac** o recurrir a un **servicio de compilación en la nube como EAS Build de Expo**.

c) ¿Qué son las **variables de entorno** del sistema operativo y por qué el emulador de Android falla cuando el Path o el Android-Home están mal configurados? diferencia entre una variable de usuario y una variable de sistema.

- Las **variables de entorno** son configuraciones del sistema operativo que almacenan información necesaria para que las aplicaciones y herramientas encuentren y utilicen determinados recursos. En el desarrollo Android, **ANDROID_HOME** indica la ubicación del Android SDK y **PATH** contiene las rutas donde el sistema busca ejecutables. Si estas variables están mal configuradas, herramientas como el emulador, adb o emulador pueden no encontrar los archivos necesarios y fallar. Una **variable de usuario** solo está disponible para un usuario específico, mientras que una **variable del sistema** está disponible para todos los usuarios del equipo.

- d) Compare el flujo de trabajo con **Expo** frente a **React Native CLI**: mencione dos ventajas y dos limitaciones de cada uno, e indique en qué situación elegiría cada opción.
- Expo ofrece un flujo de desarrollo más sencillo y rápido, con herramientas integradas para crear, desarrollar y construir aplicaciones React Native. Sus limitaciones aparecen cuando necesitan personalizaciones nativas muy específicas o cuando se utiliza Expo Go, que tiene capacidades limitadas para proyectos de producción.
 - React Native CLI ofrece mayor control sobre los proyectos nativos de Android de Android y iOS permite modificar directamente su configuración y código. Sin embargo, requiere una configuración más compleja y mayor gestión del entorno nativo.
 - Elegiría Expo para proyectos que busquen rapidez y facilidad de desarrollo, mientras que elegiría React Native CLI cuando necesite un control profundo sobre las funcionalidades nativas de Android o iOS.

PREGUNTA 2 Fundamentos de React Native

a) React Native no renderiza HTML. Complete la equivalencia entre el elemento web y el componente que cumple esa función en React Native:

En la web	En React Native
<div>	<View>
<p> o 	<Text>
	<Image>
<input>	<TextInput>
Lista larga con scroll	<FlatList>

b) Explique tres diferencias entre los estilos de React Native y el CSS tradicional. Incluya obligatoriamente el valor por defecto de `flexDirection` y por qué esa decisión tiene sentido en un dispositivo móvil.

Tres diferencias entre React Native y CSS tradicional son:

1. **Dirección de flexbox:** En CSS web `flex_direction` tiene `row` como valor por defecto, mientras que en React Native el valor por defecto es `column`, por lo que los elementos se organizan verticalmente.
2. **Unidades:** React Native utiliza principalmente valores numéricos sin unidades como `px`, mientras que CSS permite unidades como `px`, `%`, `rem`, `em`, entre otras.
3. **Definición de estilos:** CSS tradicional utiliza archivos y reglas CSS, mientras que React Native define los estilos normalmente como objetos JavaScript, mediante `stylesheet`, y utiliza en camelcase como `backgroundColor`.

c) ¿Cuál es la diferencia entre **props** y **estado**? Dé un ejemplo de un dato que debería ser `prop` y uno que debería ser `estado` dentro de una pantalla de lista de productos.

- La diferencia entre `props` y `estado` es que los `props` son datos que un componente recibe desde otro componente, generalmente desde su componente padre, mientras que el `estado` son datos que el propio componente administra y que pueden cambiar durante la ejecución.
- **Ejemplo:** En una pantalla de lista de productos, el nombre de la tienda o categoría seleccionada podría ser un `prop`, porque es información que la pantalla recibe desde otro componente. En cambio, la cantidad de productos mostrados o el producto seleccionado como favorito podría ser un `estado`, porque es información que puede cambiar según las acciones del usuario.

PREGUNTA 3 Manejo de pantallas y navegación

a) Describa para qué sirve cada tipo de navegador y proponga un caso de uso real para cada uno:

Navegador	¿Para qué sirve?	Caso de uso
Stack	Sirve para navegar entre pantallas siguiendo una estructura de pila . Cuando entras a una nueva pantalla, esta se coloca encima de la anterior y puedes regresar con el botón de atrás.	Una aplicación de compras
Tabs	Sirve para cambiar rápidamente entre las secciones principales de una aplicación , normalmente mediante pestañas ubicadas en la parte inferior.	Una aplicación de música
Drawer	Sirve para mostrar un menú lateral que se abre generalmente deslizando desde un borde de la pantalla o presionando un botón.	Una aplicación bancaria

b) ¿Qué función cumple el contenedor que envuelve toda la navegación de la app y por qué debe existir uno solo en todo el proyecto?

- El NavigationContainer es el contenedor principal de React Navigation y se encarga de gestionar el estado y el historial de navegación de la aplicación. Debe existir uno solo porque toda la aplicación debe compartir un único estado de navegación; tener varios contenedores independientes puede generar conflictos y separar la navegación en diferentes sistemas.

c) Explique cómo se **envía un parámetro** de una pantalla a otra al navegar y cómo lo **lee** la pantalla de destino. ¿Qué tipo de dato conviene enviar: el objeto completo o únicamente su identificador? Justifique.

- Para enviar un parámetro entre pantallas se utiliza navigation.navigate() pasando un objeto con params. La pantalla de destino puede leerlo mediante route.params. Por ejemplo, se puede enviar el ID de un producto navigation.navigate(“DetalleProducto”, {id: producto.id}) y recibirlo con const { id } = route.params. Es recomendable enviar únicamente el identificador en lugar del objeto completo, porque reduce la cantidad de información transferida, evita duplicar datos y permite obtener la información actualizada del objeto en la pantalla del destino.

d) Cuando el usuario navega de la pantalla A a la B y luego regresa, ¿la pantalla A se vuelve a crear desde cero o conserva su estado? Explique qué implicación tiene esa respuesta cuando la pantalla A muestra una lista que debe refrescarse tras crear un registro nuevo.

- Cuando el usuario navega de A a B y luego regresa a A, normalmente A no se crea desde cero, sino que conserva su estado mientras permanece en el stack de navegación. Por esto, si A contine una lista y en B se crea un nuevo registro, la lista de A puede seguir mostrando los dos anteriores. Para solucionarlo, se puede detectar cundo A vuelve a estar activa mediante useFocusEffect y volver a cargar los datos, actualizando así la lista.

e) ¿Qué diferencia hay en el comportamiento del *regresar* entre Android e iOS a nivel de sistema operativo, y cómo lo resuelve la librería de navegación?

- En Android existe un mecanismo de retroceso proporcionado por el sistema operativo, como el botón o gesto de “Atrás”, mientras que iOS no tiene un botón de retroceso del sistema y utiliza principalmente del botón de navegación o el gesto de deslizar desde el borde. React Navigation unifica este comportamiento mediante sus navegadores, especialmente Stack Navigator, que mantiene la pila de pantallas y permite regresar mediante Como navigation.goBack(), adaptándose a cada plataforma.

PREGUNTA 4 Configuración base de Firebase

a) Enumere en orden los pasos necesarios para dejar un proyecto de Firebase listo para ser consumido desde una app de React Native, desde la creación del proyecto en la consola hasta la inicialización dentro del código.

- 1. crear un proyecto en la consola de Firebase.
- 2. Registrar la aplicación Android o iOS dentro del proyecto.
- 3. Configurar las credenciales y descargar los archivos correspondientes, como google-services.json o GoogleService-Info.plist, según la plataforma.
- 4. Instalar las dependencias de Firebase en el proyecto React Native.
- 5. Crear el archivo de configuración con los datos del proyecto Firebase.
- 6. Inicializar Firebase en el código mediante initializeApp(firebaseConfig).
- 7. Una vez inicializando, importar e inicializar los servicios que se necesiten, como Authentication, Firestore, Storage.

b) Explique qué identifica cada una de estas claves del objeto de configuración:

Clave	¿Qué identifica?
apiKey	Identifica la clave de acceso de la aplicación a los servicios de Firebase.
projectId	Identifica de forma única el proyecto de Firebase al que pertenece la aplicación.
appId	Identifica de forma única la aplicación registrada dentro del proyecto de Firebase.
storageBucket	Identifica el almacenamiento de archivos de Firebase Storage asociado al proyecto.

c) La `apiKey` de Firebase queda visible en el paquete de la aplicación y cualquiera puede extraerla. Explique **por qué eso no constituye por sí solo una falla de seguridad** y dónde reside entonces la protección real de los datos.

- La API Key de Firebase puede quedar visible en el paquete de la aplicación porque no es una credencial secreta ni proporciona por sí sola acceso ilimitado a los Datos. Su función principal es identificar el proyecto y la aplicación. La protección real de los datos reside en las reglas de seguridad de los servicios de Firebase ,junto con la autenticación y autorización, que determina qué usuarios pueden leer, escribir o modificar la información.

- d) ¿Qué diferencia hay entre iniciar Firestore en **modo de prueba** y en **modo de producción**? ¿Qué riesgo concreto corre una app que se publica con las reglas en modo de prueba?
- En modo de prueba Firestore utiliza reglas de seguridad abiertas o temporales para facilitar el desarrollo y las pruebas, mientras que el modo de producción requiere establecer reglas que controlen explícitamente el acceso a los datos.
 - Publicar una aplicación con reglas de prueba representa un riesgo porque usuarios no autorizados podrían leer, modificar, o eliminar información de la base de datos dependiendo de las reglas configuradas.

e) Compare **Cloud Firestore** y **Realtime Database** en al menos tres aspectos, e indique cuál elegiría para la aplicación de la Parte 2 y por qué.

- Cloud Firestore y Realtime Database son bases de datos NoSQL de firebase, pero se diferencia en su estructura, consultas y escalabilidad. Cloud Firestore organiza los datos mediante colecciones y documentos, mientras que Realtime Database esta orientada a modelos de datos simples y sincronización en tiempo real con muy baja latencia. Para la aplicación de la parte 2 elegiría Cloud Firestore porque permite organizar mejor los productos, realizar consultas y filtros de manera más flexible y facilita el crecimiento de la aplicación.
- 60% · 55 minutos

Parte 2 — Componente práctico

PREGUNTA 5 Aplicación "Directorio de Contactos"

EL RETO

Construya una aplicación móvil de **tres pantallas** con navegación tipo Stack, conectada a **Cloud Firestore**, que permita listar contactos, ver el detalle de uno y registrar uno nuevo. El taller indica *qué* debe lograr y *qué* debe verse; el *cómo* es su trabajo. Está permitido consultar la documentación oficial, no soluciones completas de terceros.

PASO 1 Proyecto y entorno 8 min

- Cree un proyecto nuevo de React Native con Expo e instale las dependencias de navegación.
- Ejecútelo en un dispositivo físico con Expo Go o en un emulador.
- Modifique el texto de la pantalla inicial y confirme que el cambio se refleja sin reiniciar la aplicación.

RESULTADO ESPERADO

La aplicación abre en el dispositivo mostrando su texto personalizado. Al guardar un cambio en el editor, la pantalla se actualiza sola en menos de dos segundos. En la terminal aparece el bundler corriendo sin errores en rojo.

PASO 2 Navegación entre las tres pantallas 12 min

Configure un navegador tipo Stack con las siguientes pantallas:

Pantalla	Contenido	Navegación
Lista	Listado de contactos y un botón para agregar uno nuevo.	Es la pantalla inicial. Lleva a <i>Detalle</i> al tocar un contacto y a <i>Nuevo</i> con el botón.
Detalle	Nombre, teléfono y ciudad del contacto seleccionado.	Recibe el identificador del contacto como parámetro. Permite regresar a <i>Lista</i> .
Nuevo	Formulario con tres campos de texto y un botón de guardar.	Al guardar, regresa automáticamente a <i>Lista</i> .

El encabezado de cada pantalla debe mostrar un título en español, y en la pantalla de detalle ese título debe corresponder al nombre del contacto abierto.

RESULTADO ESPERADO

Puede recorrer las tres pantallas con datos quemados en el código, sin Firebase todavía. El botón físico de retroceso de Android y el gesto de deslizar en iOS regresan correctamente. El título del encabezado cambia según la pantalla.

PASO 3 Configuración de Firebase 12 min

- Cree un proyecto en la consola de Firebase y registre dentro de él una aplicación.
- Habilite **Cloud Firestore** y cree una colección llamada `contactos`.
- Agregue manualmente **tres documentos** de prueba desde la consola, cada uno con los campos nombre, teléfono y ciudad.
- Instale el SDK de Firebase en el proyecto y centralice la inicialización en un **único archivo** de configuración que el resto de la app importe.
- Los valores de configuración no deben quedar escritos directamente en el código versionado.

RESULTADO ESPERADO

La aplicación arranca sin errores de inicialización de Firebase y en la consola de Firestore se ven los tres documentos creados dentro de la colección `contactos`.

ADVERTENCIA

Si sube al repositorio el archivo con sus credenciales de Firebase, el criterio de seguridad de la rúbrica se califica en cero. Use un archivo de variables de entorno excluido por `.gitignore` y publique en su lugar un archivo de ejemplo con los nombres de las variables y valores vacíos.

PASO 4 Lectura y escritura de datos 18 min

- La pantalla **Lista** obtiene los contactos desde Firestore y los muestra en una lista con desplazamiento eficiente, no en un contenedor de scroll simple.
- Mientras los datos cargan se muestra un indicador de carga, y si la colección está vacía se muestra un mensaje que lo indique.
- La pantalla **Detalle** muestra los datos del contacto que corresponde al identificador recibido como parámetro.
- La pantalla **Nuevo** guarda un documento en Firestore con los tres campos y no permite guardar si algún campo está vacío.
- Tras guardar, la pantalla **Lista** debe reflejar el contacto nuevo sin necesidad de cerrar y volver a abrir la aplicación.

RESULTADO ESPERADO

Al abrir la app aparecen los tres contactos creados desde la consola. Al registrar uno nuevo desde el formulario, este aparece en la lista al regresar y también queda visible en la consola de Firebase. Al intentar guardar con un campo vacío, la aplicación lo impide y avisa al usuario.

PASO 5 Verificación final 5 min

- ☐ La app corre en dispositivo o emulador sin pantallas rojas de error.
- ☐ Las tres pantallas navegan en ambos sentidos y los títulos son correctos.
- ☐ El detalle recibe y usa el parámetro enviado desde la lista.
- ☐ Los datos provienen de Firestore, no de un arreglo quemado en el código.
- ☐ Un contacto creado desde la app aparece en la consola de Firebase.
- ☐ Las credenciales no están versionadas en el repositorio.

Retos opcionales bono de 0.5

1. Reemplace la consulta única por una **suscripción en tiempo real**: un cambio hecho desde la consola de Firebase se refleja en la app sin tocar nada.
2. Agregue la opción de **eliminar** un contacto desde la pantalla de detalle, con confirmación previa.
3. Cambie el navegador Stack por una combinación de **Tabs** con dos secciones (Contactos y Acerca de), manteniendo la navegación de detalle dentro de la primera.
4. Ajuste las **reglas de seguridad** de Firestore para que la colección sea de lectura pública pero de escritura restringida, y documente la regla en el `RESPUESTAS.md`.

Entregable

Se entrega un único enlace: la URL del repositorio público en GitHub, publicada en la actividad correspondiente de Virtual TdeA.

El repositorio debe contener:

- El código completo de la aplicación, sin la carpeta `node_modules`.
- Un archivo `RESPUESTAS.md` con las respuestas de las preguntas 1 a 4.
- Un archivo `README.md` con su nombre completo, el sistema operativo en el que trabajó y los pasos para ejecutar el proyecto.
- Un archivo de ejemplo con los nombres de las variables de entorno de Firebase, sin valores.
- Un `.gitignore` que excluya `node_modules` y el archivo de credenciales.
- Una carpeta `capturas/` con cuatro imágenes: las tres pantallas de la app y la consola de Firestore mostrando el contacto creado desde el dispositivo.

Los repositorios privados o los enlaces que no abran no se califican. Verifique la URL en una ventana de incógnito antes de publicarla en Virtual TdeA.

Rúbrica de evaluación

Criterio	Peso
Preguntas 1 a 4: claridad, corrección conceptual y justificación de las respuestas	40%
El proyecto ejecuta sin errores en dispositivo o emulador	10%
Navegación: tres pantallas, paso de parámetros y títulos correctos	20%
Firebase: inicialización centralizada, lectura y escritura funcionando	20%
Manejo seguro de credenciales y repositorio completo y ordenado	10%
Total	100%

Documentación recomendada: la guía de inicio de Expo, la sección *Fundamentals* de React Navigation y la guía *Get started with Cloud Firestore* en la documentación oficial de Firebase.